



Manual Técnico

Instalación, Configuración y Despliegue

Canal Ético de Denuncias

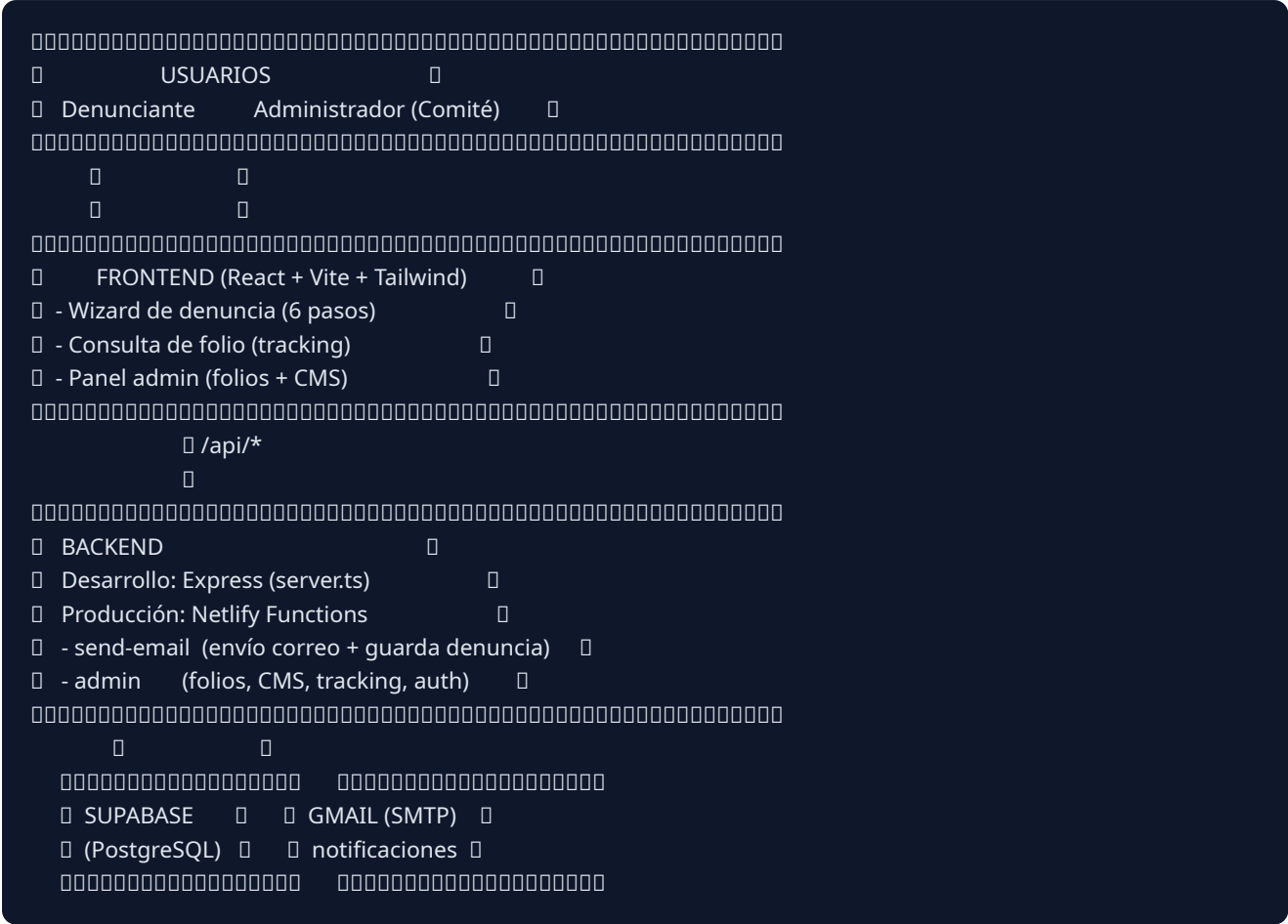
Guía dirigida al equipo técnico / responsable de TI

Contenido

1. Arquitectura general del sistema
2. Tecnologías utilizadas (stack)
3. Requisitos previos
4. Instalación local (desarrollo)
5. Configuración de Supabase (base de datos)
6. Variables de entorno
7. Configuración del correo (Gmail)
8. Despliegue en producción (Netlify)
9. Estructura de la base de datos
10. Mantenimiento y respaldos
11. Seguridad

1. Arquitectura general del sistema

El sistema es una aplicación web de página única (SPA) construida con React, respaldada por una base de datos en la nube (Supabase / PostgreSQL) y un backend que funciona tanto como servidor Express (en desarrollo) como funciones serverless (en producción con Netlify).



2. Tecnologías utilizadas (stack)

Capa	Tecnología
Frontend	React 19, Vite 6, Tailwind CSS 4, React Router 7
Lenguaje	TypeScript 5.8
Backend (dev)	Express 4, ejecutado con <code>tsx</code>
Backend (prod)	Netlify Functions (serverless)
Base de datos	Supabase (PostgreSQL) vía <code>@supabase/supabase-js</code>
Autenticación admin	JWT (<code>jsonwebtoken</code>) en cookie httpOnly
Contraseñas	Hash con <code>bcryptjs</code>
Correo	Nodemailer (Gmail SMTP)
Íconos / UI	lucide-react, react-hot-toast

3. Requisitos previos

- **Node.js** (versión 18 o superior).
- Una cuenta de **Supabase** (el plan gratuito es suficiente).
- Una cuenta de **Gmail** con contraseña de aplicación para el envío de correos.
- Una cuenta de **Netlify** para el despliegue en producción (opcional).
- Git instalado (para clonar el repositorio).

4. Instalación local (desarrollo)

1 Clonar el repositorio

```
git clone https://github.com/juanhernandez11/canal_de_denuncias_arh.git
cd canal_de_denuncias_arh
```

2 Instalar dependencias

```
npm install
```

3 Configurar variables de entorno (ver sección 6)

```
cp .env.example .env.local
# editar .env.local con tus credenciales
```

4 Ejecutar la aplicación

```
npm run dev
```

La aplicación quedará disponible en <http://localhost:3000> .

Scripts disponibles

Comando	Descripción
<code>npm run dev</code>	Servidor de desarrollo (Express + Vite).
<code>npm run build</code>	Genera el build de producción en <code>dist/</code> .
<code>npm run preview</code>	Previsualiza el build de producción.
<code>npm run lint</code>	Verificación de tipos (<code>tsc --noEmit</code>).
<code>npm run clean</code>	Elimina la carpeta <code>dist/</code> .

5. Configuración de Supabase (base de datos)

- 1 Crea un proyecto nuevo en supabase.com.
- 2 En el dashboard, abre **SQL Editor**.
- 3 Copia y pega el contenido del archivo `supabase/schema.sql` del proyecto y presiona **Run**. Esto crea las tablas `admins`, `denuncias` y `content_blocks`, siembra el contenido editable por defecto y habilita RLS.

El usuario administrador por defecto (`admin` / `admin123`) **no** se crea en el SQL; lo crea automáticamente el servidor la primera vez que se conecta a Supabase.

6. Variables de entorno

Copia `.env.example` a `.env.local` y completa los siguientes valores:

Variable	Descripción
<code>EMAIL_USER</code>	Cuenta de Gmail para enviar notificaciones.
<code>EMAIL_PASS</code>	Contraseña de aplicación de Gmail (no la contraseña normal).
<code>SUPABASE_URL</code>	Project Settings ▢ API ▢ Project URL.
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Project Settings ▢ API ▢ service_role (SECRETA).
<code>JWT_SECRET</code>	Cadena larga y aleatoria para firmar las sesiones del admin.

❑ La **service_role key** es secreta. Solo se usa en el backend. Nunca la expongas en el cliente ni la subas al repositorio. El archivo `.env.local` está excluido en `.gitignore`.

7. Configuración del correo (Gmail)

Para que el sistema envíe correos de notificación y confirmación:

- 1 Activa la **verificación en dos pasos** en la cuenta de Gmail.
- 2 Genera una **contraseña de aplicación** desde la configuración de seguridad de Google.
- 3 Usa esa contraseña de aplicación en la variable `EMAIL_PASS` (no uses tu contraseña normal de Gmail).

Cuando se envía una denuncia, el sistema notifica al comité y, si el denunciante se identificó, también le envía un correo de confirmación con su folio.

8. Despliegue en producción (Netlify)

El proyecto incluye `netlify.toml` preconfigurado. La capa de datos usa la API REST de Supabase, por lo que funciona en entornos serverless.

- 1 Conecta el repositorio de GitHub a Netlify.
- 2 Netlify detectará la configuración: comando de build `npm run build`, carpeta de publicación `dist`, funciones en `netlify/functions`.
- 3 En Netlify ❑ Site settings ❑ Environment variables, define las **mismas variables de entorno** de la sección 6.
- 4 Despliega. Netlify redirige automáticamente: `/api/send-email`, `/api/admin/*`, `/api/content` y `/api/folios/*` a las funciones serverless correspondientes.

9. Estructura de la base de datos

Tabla: `admins`

Campo	Tipo	Descripción
id	bigint	Identificador único.
username	text	Nombre de usuario (único).
password_hash	text	Contraseña cifrada con bcrypt.
created_at	timestamp	Fecha de creación.

Tabla: `denuncias`

Campo	Descripción
folio	Identificador único de la denuncia (ARH-2026-XXXXX).
estatus	recibida / en_revision / en_investigacion / resuelta / desestimada.
tipo, empresa, centro, modo	Datos generales de la denuncia.
denunciante_nombre, denunciante_correo	Datos del denunciante (si se identificó).
descripcion	Descripción de los hechos.
payload_json	JSON completo con todos los datos del formulario.
notas_admin	Notas internas del comité.
created_at, updated_at	Fechas de creación y actualización.

Tabla: `content_blocks`

Almacena los textos editables del sitio (CMS): título, subtítulo, aviso de privacidad, pie de página, correo de contacto, etc.

10. Mantenimiento y respaldos

- **Respaldos:** Supabase realiza respaldos automáticos. Se recomienda además exportar periódicamente la tabla `denuncias` desde el SQL Editor.
- **Actualización de dependencias:** Ejecuta `npm outdated` y actualiza con precaución, verificando con `npm run lint` y `npm run build`.
- **Monitoreo:** Revisa los logs de Netlify Functions y de Supabase ante cualquier incidencia.

11. Seguridad

Aspecto	Implementación
Autenticación admin	JWT firmado con <code>JWT_SECRET</code> , en cookie httpOnly.
Contraseñas	Cifradas con bcrypt (nunca en texto plano).
Base de datos	RLS habilitado; el backend usa service_role (acceso controlado).
Rutas admin	Protegidas con middleware <code>requireAuth</code> .
Tracking público	Solo expone estatus y fecha; nunca datos sensibles.
Secretos	En variables de entorno, fuera del repositorio.

Acciones obligatorias antes de producción:

- Cambiar la contraseña del usuario `admin` por defecto.
- Definir un `JWT_SECRET` largo y aleatorio.
- Verificar que `.env.local` no esté versionado.